STAFF REVIEW · H446 120 source models

Separate from the eight-page pupil workbook. This copy is in a public repository and is NOT access controlled. Check technical correctness before use.

Printed question mapping: Booklet Q1 = original online Q1; Booklet Q2 = original online Q2; Booklet Q3 = original online Q3

Do Now — source answers

1. Identification, assessment and mitigation of risk.

2. Its element locations can be calculated from base address and index.

3. Sum 0 and carry-out 1.

4. 10.

I Do — source worked steps

1. Model answer — At n=1 total 0; n=2 total 2; n=3 total 2; n=4 total 6; output 6. Record n and total at each loop step. — Check: link each claim to the scenario and use precise OCR terminology.

2. Spot the stopping condition — n == 0 is the base case : return 0 immediately. Otherwise SumTo(n - 1) invokes the same method with a smaller argument. Assume n is non-negative.

3. Trace calls before returns — SumTo(3) waits for SumTo(2) , then SumTo(1) , then SumTo(0) . The first three calls remain active until the base case returns.

4. Follow the returns in reverse order — SumTo(0)=0 ; then SumTo(1)=1 , SumTo(2)=3 , SumTo(3)=6 . An iterative alternative would use a loop and running total; recursion instead retains pending calls.

We Do — guided CHECK

1. What must we decide?
Identify the inputs, relevant facts and exact output or judgement required.
Work through the method
Use the middle stage of the visual to follow the process and state each intermediate result.
Shared worked answer
Progress: initial 0; after 3 → 3; after 8 → 8; after 5 stays 8; final output 8. The initial zero fails if all values are negative; use first element instead.

2. Find the base case
Which inputs return 1 immediately? What argument does Factorial(3) pass into its next call?
Predict the order
Factorial(3) → Factorial(2) → Factorial(1) . The final call returns 1; earlier calls then resume in reverse order.
Unwind to the result
Factorial(2)=2×1=2 ; Factorial(3)=3×2=6 . A loop with result=1 and repeated multiplication could also produce 6. Neither approach automatically solves negative-input validation.

You Do — source mark schemes

1. Odd i values 1,3,5 cause increments; count progresses 1,2,3; output 3. Credit correctly labelled trace columns and output.

2. Zero exceeds all negative inputs [2], result incorrectly remains 0 [2], initialise from first actual element and iterate remainder [2].

3. Correct descending call arguments 4,3,2,1,0 [2]; base case n=0 returns 0 [2]; unwinding returns 1,3,6,10 [2]; iteration uses a loop/running total rather than pending function calls [2].

Exit Ticket — source answer

1. False. It records state changes as instructions execute.